EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 13

Multilayer Systems, Reusable Components and Architectural Design

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 13 of 16

Contents

1. Week 13 Introduction 4
2. Learning Outcomes 4
PART A — WHAT IS SOFTWARE ARCHITECTURE? 5
3. A Simple Question 5
PART B — MULTILAYER ARCHITECTURE 5
4. What Is a Layer? 5
PART C — THE PRESENTATION LAYER 6
5. Presentation 6
PART D — THE APPLICATION / SERVICE LAYER 6
6. Application Services 6
PART E — THE DOMAIN / BUSINESS LAYER 7
7. Domain and Business Rules 7
PART F — THE DATA ACCESS LAYER 7
8. Data Access 7
PART G — THE COMPLETE MULTILAYER MODEL 8
PART H–J — SEPARATION OF CONCERNS, COHESION AND COUPLING 8
9. Separation of Concerns 8
10. Cohesion and Coupling 9
PART K — WHAT IS A SOFTWARE COMPONENT? 10
11. Component vs Class 10
PART L — COMPONENT INTERFACES 11
12. Why Components Need Interfaces 11
PART M & N — REUSABLE AND ADAPTABLE COMPONENTS 11
13. Reusable Components 11
14. Adaptable Components 12
15. Reuse and Inheritance 13
PART P — COMPONENT DIAGRAM 13
16. UML Component Diagrams 13
PART Q & R — PACKAGES AND PACKAGE vs COMPONENT 14

17. Packages 14
PART S & T — DEPENDENCIES AND DEPENDENCY DIRECTION 14
18. Dependencies 14
PART U & V — BENEFITS AND LIMITATIONS OF MULTILAYER ARCHITECTURE 15
19. Why Multilayer Architecture Matters 15
20. Limitations — More Layers Is Not Always Better 16
PART W & X — REUSE AND ADAPTABILITY CASE STUDIES 16
21. Component Reuse and Adaptability 16
PART Y & Z — COMPONENT DIAGRAM EXERCISE AND NOTATION 17
22. Exercise and Notation 17
PART AB — DEPLOYMENT THINKING 17
23. Logical vs Deployment Architecture 17
PART AC & AD — REUSABILITY DESIGN PRINCIPLES 17
24. Principles and a Warning 17
25. Tutorial and Practical 18
26. Common Student Confusions 18
27. Week 13 Summary 19
28. Examination-Style Questions 19
29. References and Further Reading 20

1. Week 13 Introduction

Week 13 addresses one of the explicit learning outcomes of the syllabus: **"Express the value of designing a multilayer system built of reusable and adaptable components."** Earlier weeks asked what objects we need, how they behave and how they interact; Weeks 10–12 turned the analysis into an integrated design. Week 13 now asks a higher-level question: **how should we organise the software so that it is easier to maintain, reuse, replace and adapt?**

That leads us to architectural layers, components, interfaces, dependencies, reusable and adaptable components, separation of concerns, coupling, cohesion, package organisation, component diagrams and deployment thinking. UML provides structural diagram types — including class, component, package and deployment diagrams — which are particularly relevant this week.

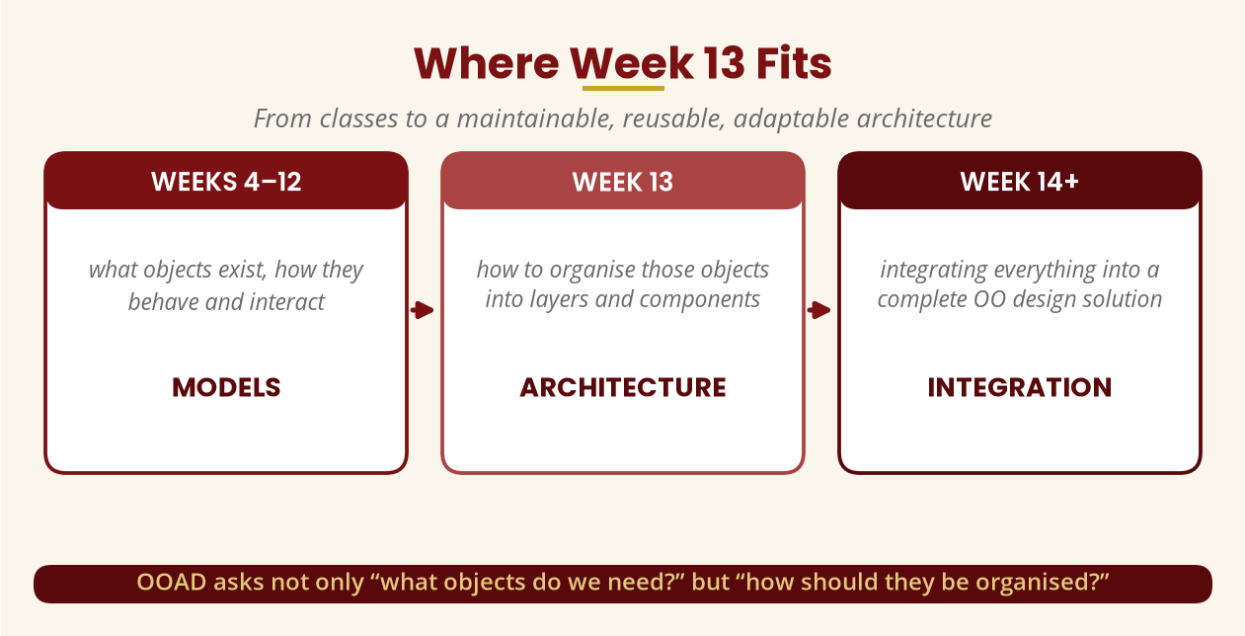

Figure 1 — Where Week 13 fits

2. Learning Outcomes

By the end of this week, a student should be able to:

- Explain software architecture in OOAD, and distinguish architecture from class design.
- Explain multilayer architecture and identify common software layers.
- Explain separation of concerns and the relationship between layers and objects.
- Define a software component and explain component interfaces.
- Explain reusable and adaptable components.
- Explain coupling and cohesion in architectural design and identify inappropriate dependencies.
- Draw a basic UML component diagram and explain package organisation.
- Explain the benefits and limitations of multilayer architecture, and integrate component/architectural thinking into an OOAD solution.

PART A — WHAT IS SOFTWARE ARCHITECTURE?

3. A Simple Question

A university tells a programmer: *"Build us an online registration system."* The programmer could create hundreds of classes — but another question remains: **how should those classes be organised?** Should the class that displays a web page also validate passwords, calculate fees, talk directly to the database, send emails and create registrations? It could be programmed that way, but it would become difficult to understand and maintain. So OOAD asks not only *what objects do we need?* but *how should those objects and components be organised?*

Software architecture is the high-level organisation of a software system, including its major parts, their responsibilities, relationships and interactions — the overall plan of the software. **Class design** asks what classes we need (Student, Course, Registration, Payment); **architecture** asks how they should be organised (Presentation Layer -> Registration Service -> Registration Domain -> Repository -> Database). **Architecture is broader than individual class design.**

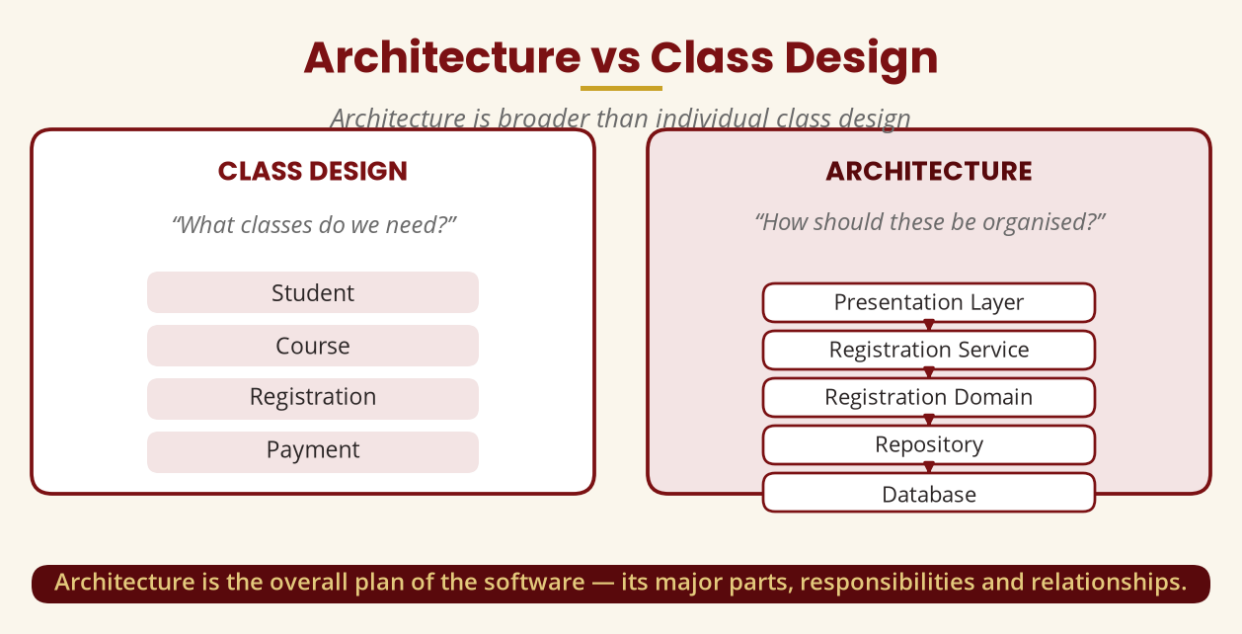

Figure 2 — Architecture vs class design

PART B — MULTILAYER ARCHITECTURE

4. What Is a Layer?

A **layer** is a logical grouping of related responsibilities — think of a school building: third floor administration, second floor classrooms, first floor library, ground floor reception. Software can be divided into layers in the same way. A common conceptual structure is the **basic multilayer architecture**: **Presentation -> Application/Service -> Domain/Business -> Data Access/Infrastructure -> Database**. Understand every layer rather than simply memorising the names.

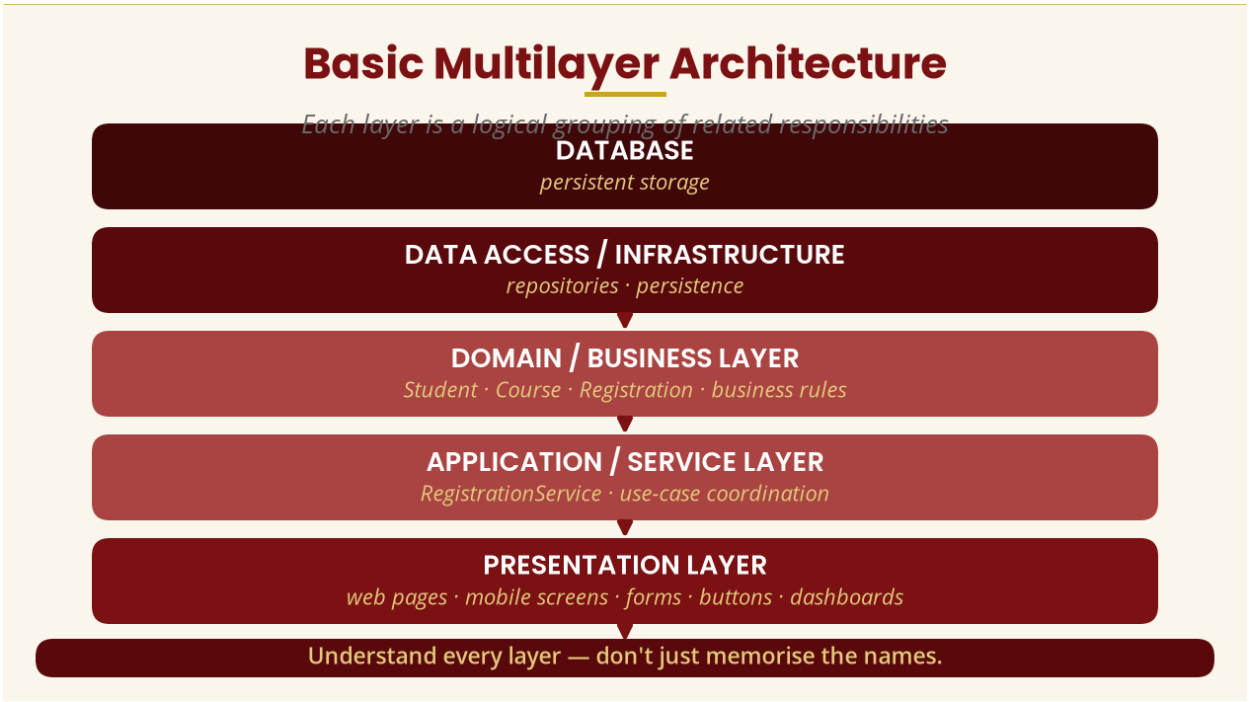

Figure 3 — Basic multilayer architecture

PART C — THE PRESENTATION LAYER

5. Presentation

The **presentation layer** interacts with the user — web pages, mobile screens, desktop windows, forms, buttons, menus, dashboards. It should primarily **display information, collect user input, do basic formatting, send requests to application services and display responses**. It should not normally contain all the business rules: don't scatter `if course.capacity > registeredStudents` throughout every interface, because a web, Android, iOS and desktop application would each duplicate the same logic — changing one rule would require changing several applications.

PART D — THE APPLICATION / SERVICE LAYER

6. Application Services

The **application layer** coordinates application operations. `RegistrationService` provides `registerStudent()`, `dropCourse()` and `getRegisteredCourses()`; it receives a request from the presentation layer and coordinates the necessary objects. When the student clicks **Register**: Web Page -> RegistrationController -> RegistrationService -> Course -> Student -> Registration. The UI does not need to know every internal detail.

PART E — THE DOMAIN / BUSINESS LAYER

7. Domain and Business Rules

The **domain layer** holds the core business concepts and rules — Student, Course, Registration, Programme, Semester — plus rules such as *"a student cannot register without satisfying the prerequisite"* and *"a course cannot accept registrations beyond capacity."* Centralising the rules matters: if the university changes *"maximum 6 courses"* to *"maximum 8 courses"*, a centralised rule is changed in one place, whereas a duplicated rule would have to be changed in the website, mobile app, desktop app and reports.

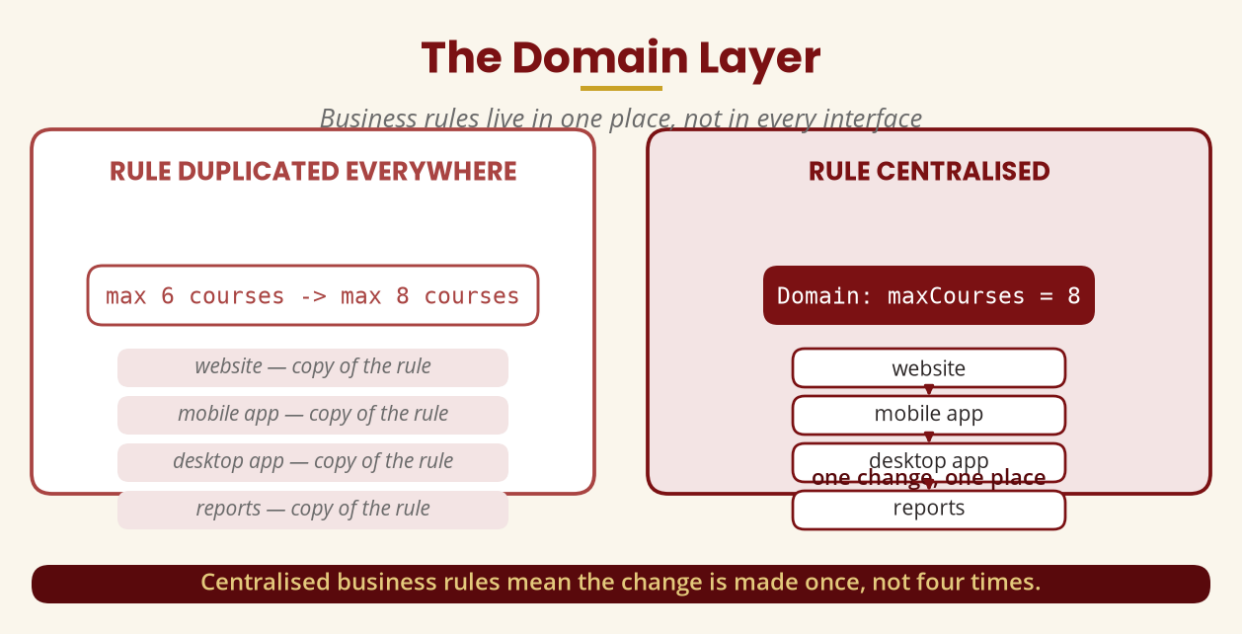

Figure 4 — Centralised business rules

PART F — THE DATA ACCESS LAYER

8. Data Access

The **data access layer** handles communication with data storage — `StudentRepository`, `CourseRepository`, `RegistrationRepository`. The service asks *"give me this student's registrations"* and the repository handles the mechanics. If every class accessed the database directly (Student -> Database, Course -> Database, Payment -> Database), changing database technology would affect many classes; routing through repositories reduces direct dependencies.

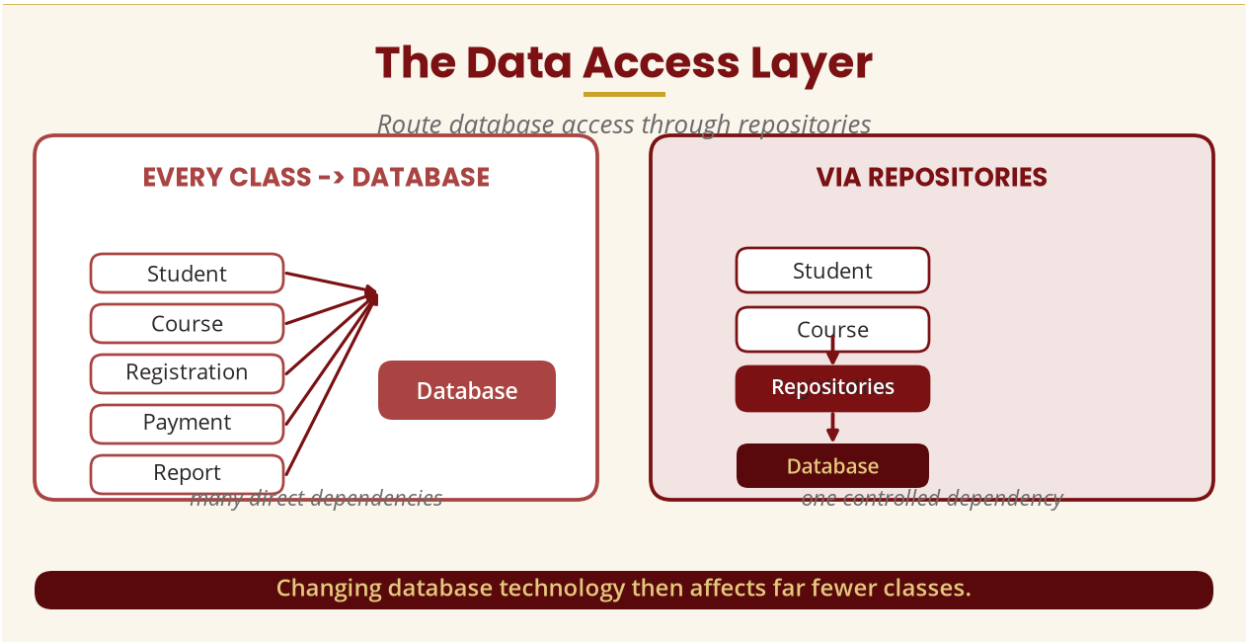

Figure 5 — The data access layer

PART G — THE COMPLETE MULTILAYER MODEL

The university registration example: **Presentation** (Web UI / Mobile UI / Desktop UI) -> **Application** (RegistrationService, AuthenticationService, CourseService) -> **Domain** (Student, Course, Registration, Programme) -> **Data Access** (StudentRepository, CourseRepository, RegistrationRepository) -> **Database**. This is the central example of the lecture.

PART H–J — SEPARATION OF CONCERNS, COHESION AND COUPLING

9. Separation of Concerns

Separation of concerns means different parts of the system concentrate on different responsibilities: Presentation -> user interaction; Application -> use-case coordination; Domain -> business rules; Data Access -> persistence. A single `StudentRegistration.java` containing displayHTML(), validatePassword(), connectDatabase(), calculateFees(), registerCourse(), sendEmail(), generatePDF() and checkPrerequisite() has too many responsibilities and becomes difficult to test, modify, reuse and understand. Separate them into StudentController, RegistrationService, Course, Registration, PaymentService, EmailService and RegistrationRepository.

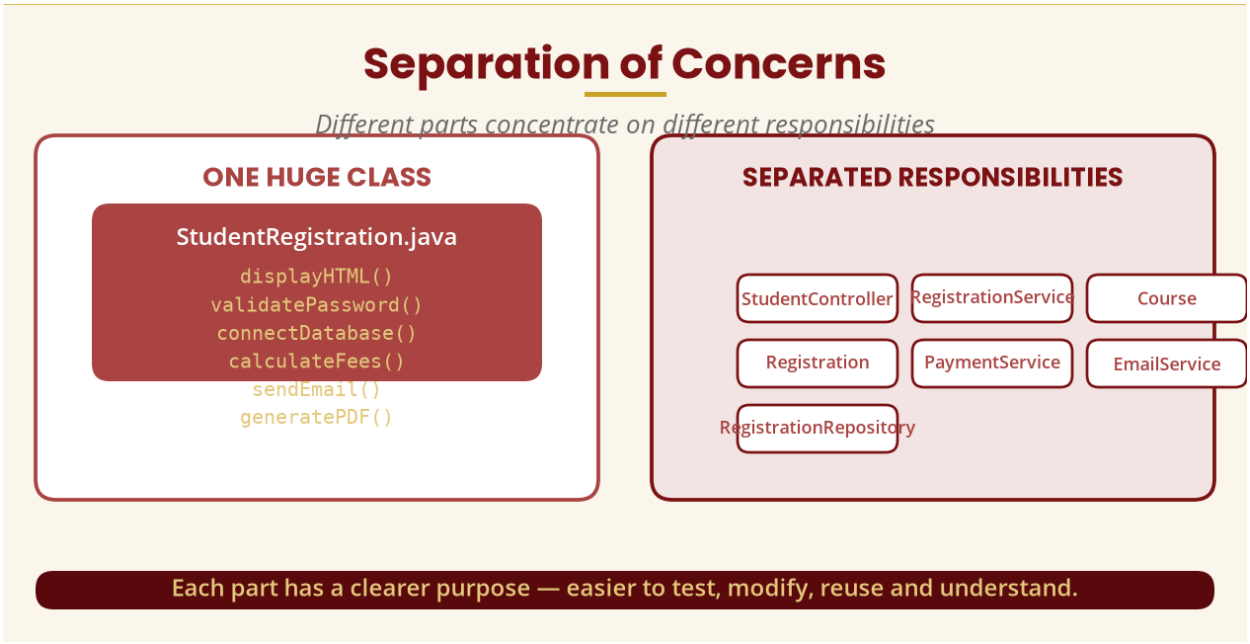

Figure 6 — Separation of concerns

10. Cohesion and Coupling

Cohesion asks how closely related the responsibilities within one class or component are — a RegistrationService containing registration operations is cohesive, while one also handling payroll, image editing, email and weather forecasting is not. A good class should have a clear reason for existing. **Coupling** describes how strongly one element depends on another — a web page that knows the database technology, table names and SQL is strongly coupled; routing through RegistrationService -> RegistrationRepository -> Database provides a cleaner separation. The principle: **aim for high cohesion and appropriately low coupling** — "keep related things together and unnecessary dependencies apart."

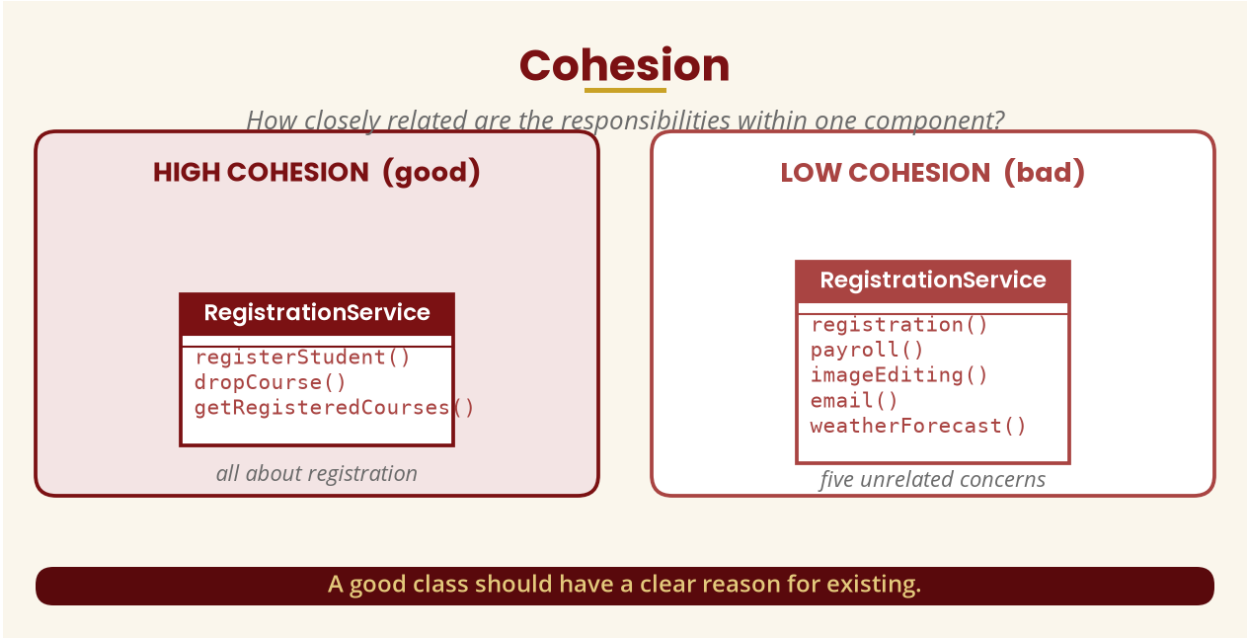

Figure 7 — Cohesion

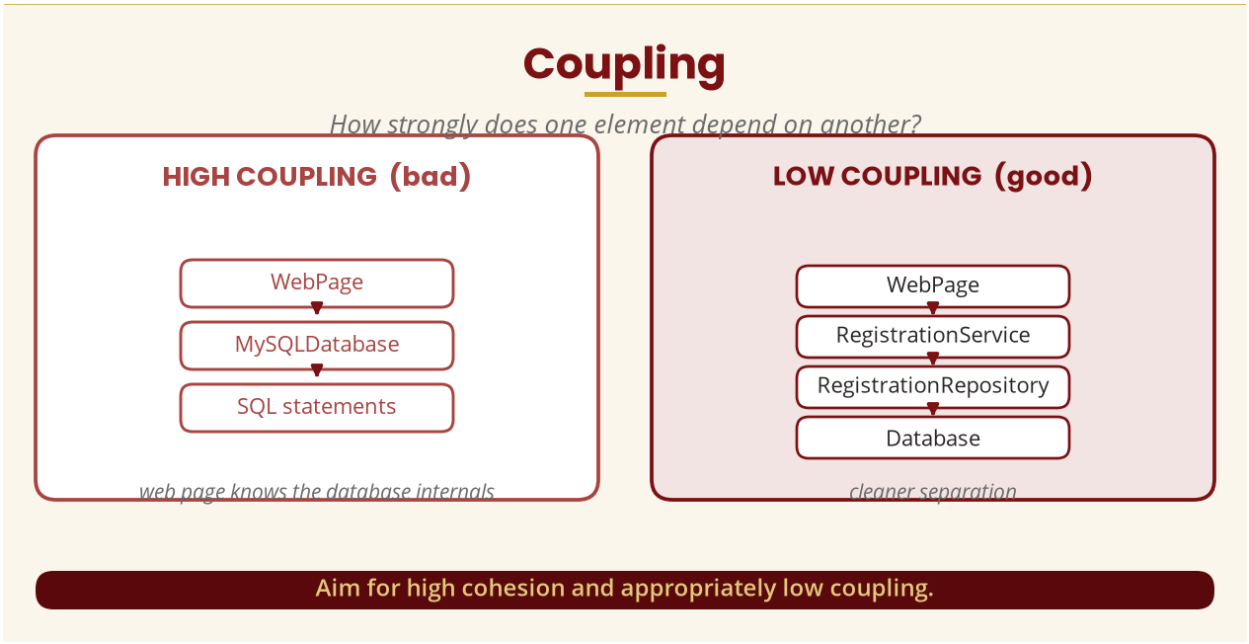

Figure 8 — Coupling

PART K — WHAT IS A SOFTWARE COMPONENT?

11. Component vs Class

A **software component** is a modular part of a system that encapsulates functionality and interacts with other parts through defined interfaces. A **class** is a finer-grained modelling/programming element with attributes, operations and relationships; a **component** is a larger modular unit — Student, Course and Registration classes can collectively contribute to a Registration Component. Think of a car: the engine is a component, but it contains many parts. A Payment Component may internally contain PaymentService, Payment, PaymentRepository and PaymentValidator, while the external system does not need to know every internal class.

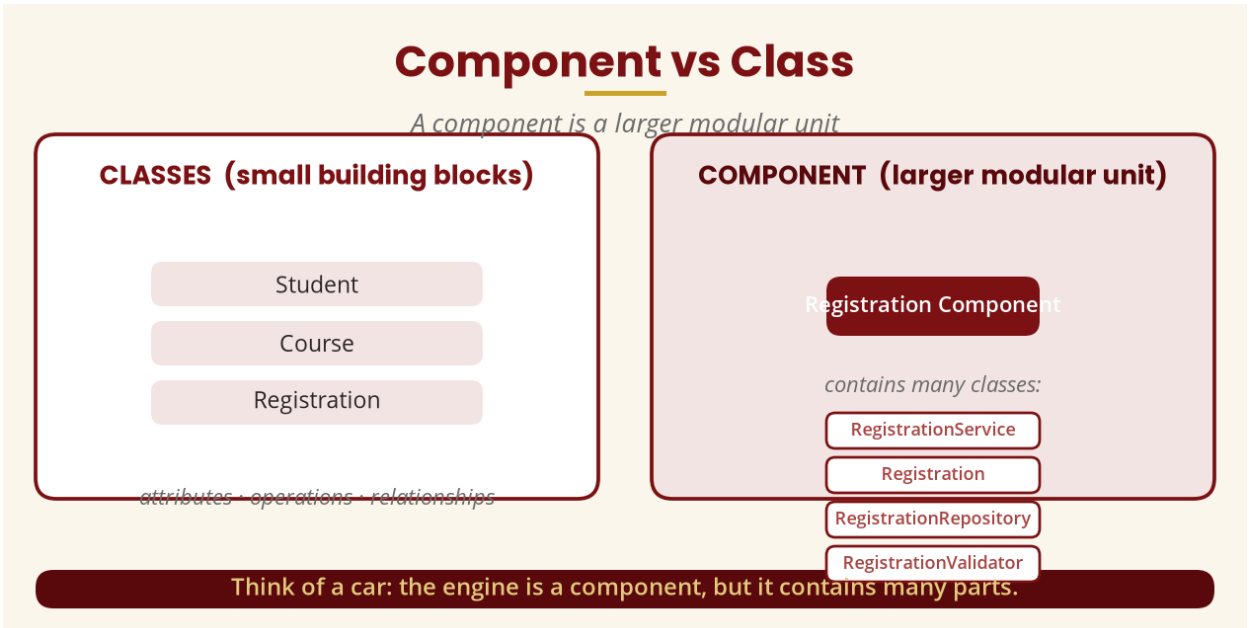

Figure 9 — Component vs class

PART L — COMPONENT INTERFACES

12. Why Components Need Interfaces

The registration system wants `processPayment()` but doesn't care whether payment goes through bank, mobile money or card — it cares about the service it can request. A **Payment Interface** (`processPayment()`, `refundPayment()`) implemented by MobileMoney, BankGateway and CardGateway lets the system replace one implementation with another without rewriting every client. If both MobileMoneyPayment and BankPayment implement PaymentGateway, the application works with the abstraction — an important OO design principle.

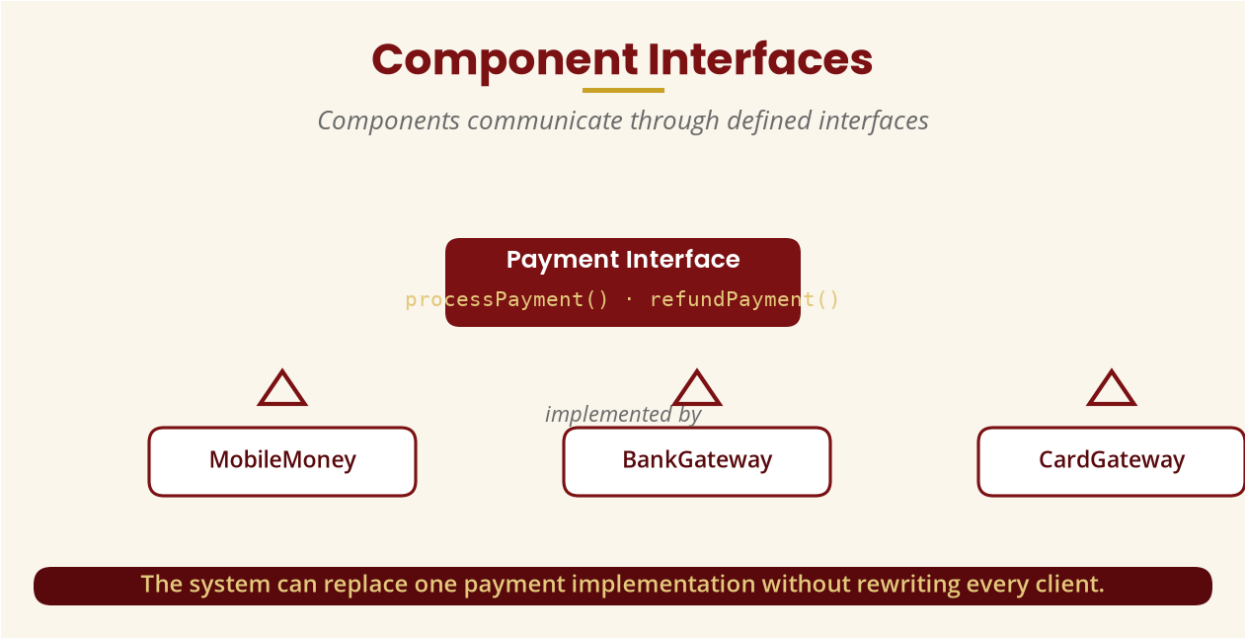

Figure 10 — Component interfaces

PART M & N — REUSABLE AND ADAPTABLE COMPONENTS

13. Reusable Components

A **reusable component** is designed so it can be used in more than one context without rebuilding it — an **Authentication Component** (login(), logout(), changePassword(), resetPassword()) can serve the University Portal, Library System, Student Mobile App and Staff Portal. Reuse can reduce development effort and duplication, improve consistency, ease maintenance and speed up development — **but reuse is not automatically beneficial**: a component that is extremely difficult to adapt may cost more to reuse than to rebuild.

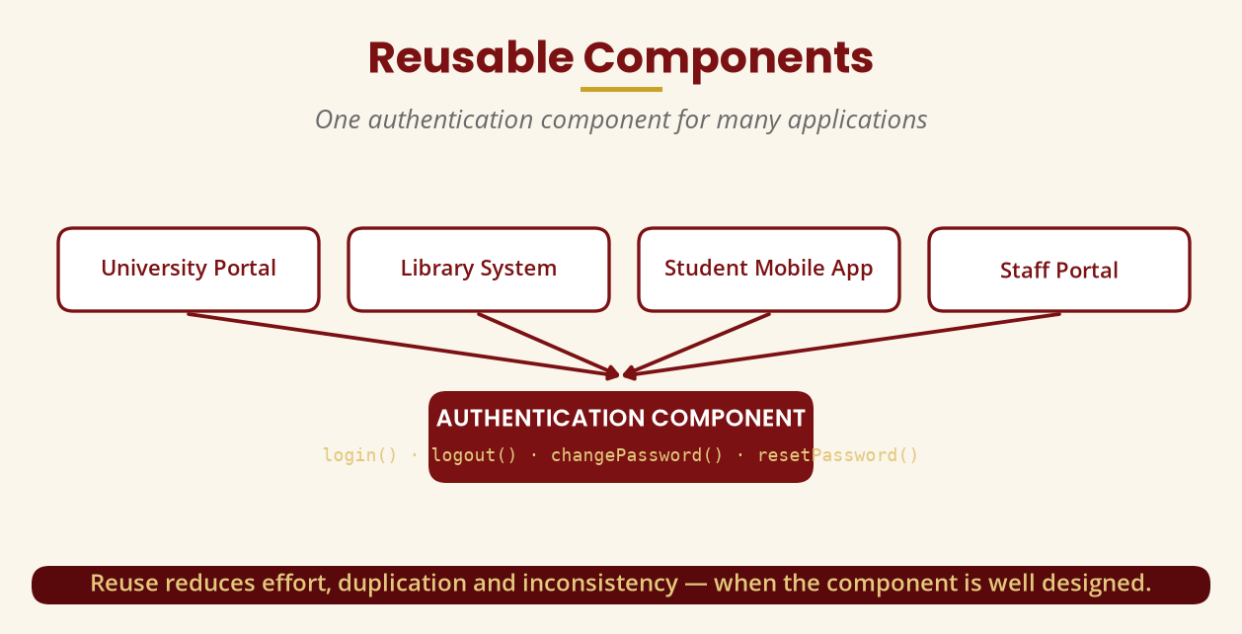

Figure 11 — Reusable components

14. Adaptable Components

An **adaptable component** can be modified, configured or extended to work in different situations. An `EmailNotification.java` hard-coded to Gmail, HTML, a specific sender and message format must be rewritten when SMS is needed; a `NotificationService` with EmailNotification, SMSNotification and PushNotification can support additional methods more easily.

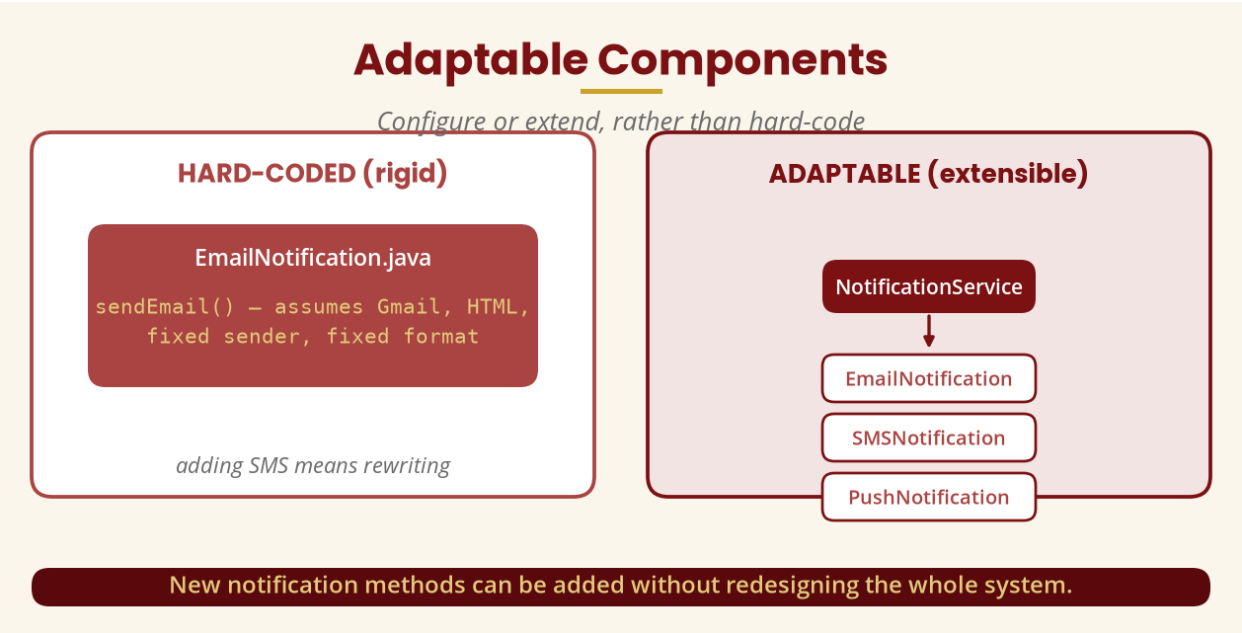

Figure 12 — Adaptable components

15. Reuse and Inheritance

Inheritance is **not** always the best way to reuse code. “Inheritance means reuse” does not imply “whenever I want reuse, I should use inheritance.” Inheritance represents an **IS-A** relationship (Administrator IS-A User), but RegistrationService IS-A Database makes no sense. Sometimes **composition** is better: RegistrationService *uses* NotificationService — it isn't a type of notification service. Other reuse approaches include composition, interfaces, services, libraries, frameworks and configurable components.

PART P — COMPONENT DIAGRAM

16. UML Component Diagrams

A **UML component diagram** shows the organisation and relationships of software components — OMG identifies it as one of UML's structural diagram types. A simplified university example: Web Application -> Authentication Component -> Registration Component -> (Course Component, Notification Component) -> Database Component. Students should learn to represent the same idea using proper UML component notation in a CASE tool.

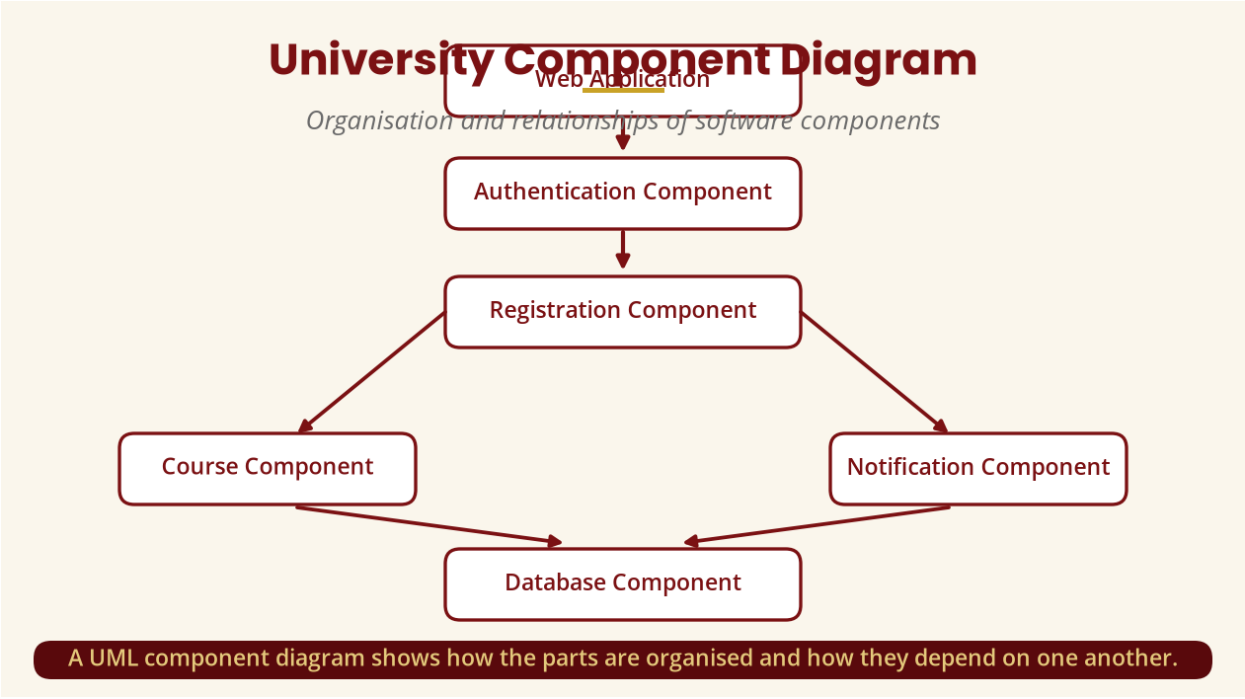

Figure 13 — University component diagram

PART Q & R — PACKAGES AND PACKAGE vs COMPONENT

17. Packages

A **UML package** groups related model elements — OMG defines a package as a namespace that can contain or import model elements. A university system might group elements into Authentication, Academic, Registration, Finance, Notification and Reporting packages. **Package vs component:** a package primarily helps **organise** model elements (Academic -> Student, Course, Programme), while a component represents **modular functionality** with interfaces (RegistrationComponent). They can be used together — a package is organisation/grouping; a component is modular functionality.

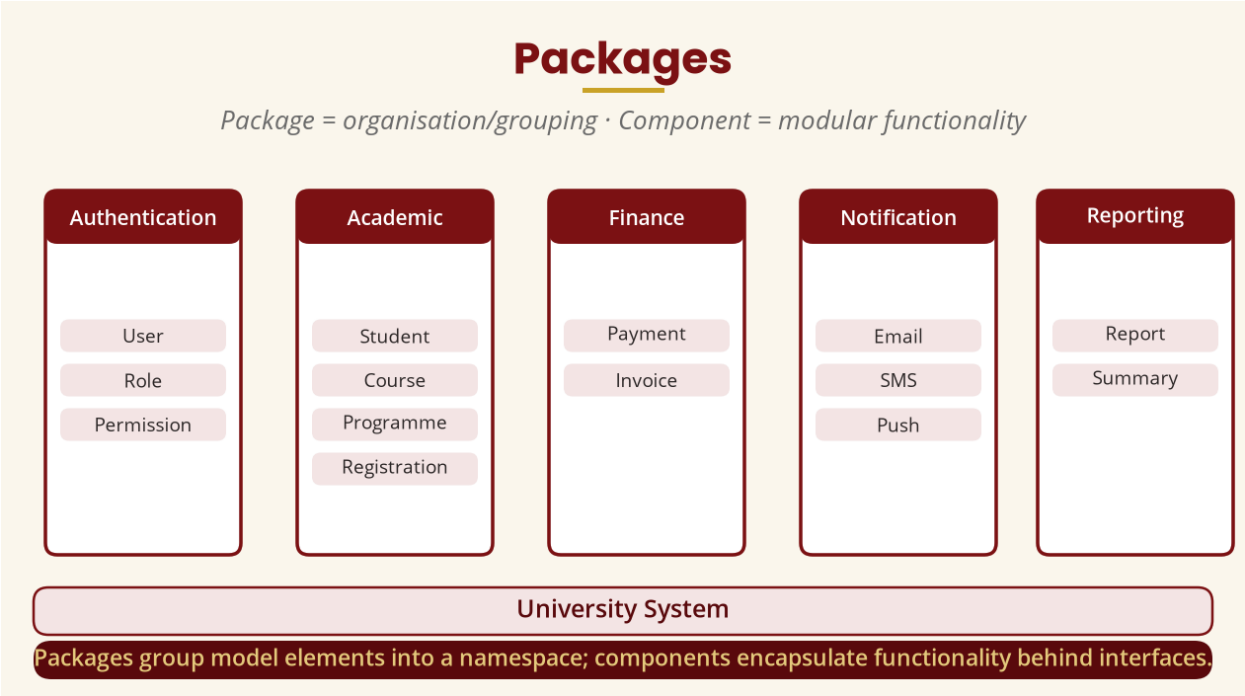

Figure 14 — Packages

PART S & T — DEPENDENCIES AND DEPENDENCY DIRECTION

18. Dependencies

A **dependency** exists when one element relies on another — RegistrationService depends on RegistrationRepository. Too many dependencies (A -> B, C, D, E, F, G) means changing any of them may affect A. A simple architectural rule is that dependencies should move in a **controlled direction**: Presentation -> Application -> Domain -> Infrastructure — not every part depending on every other part.

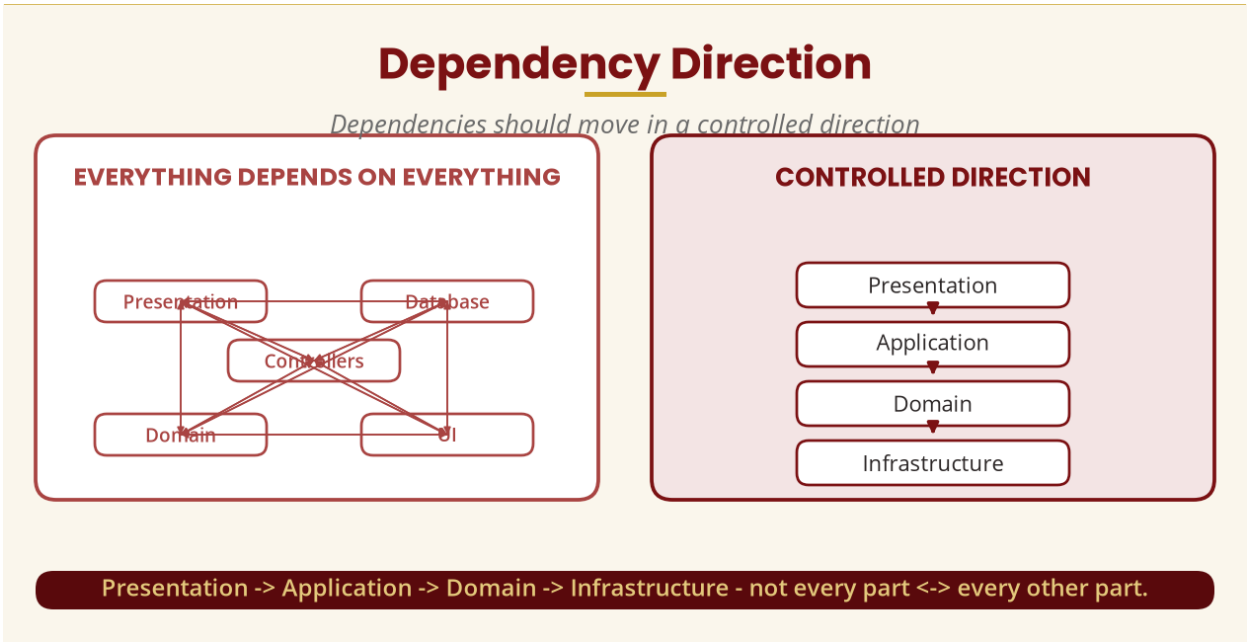

Figure 15 — Dependency direction

PART U & V — BENEFITS AND LIMITATIONS OF MULTILAYER ARCHITECTURE

19. Why Multilayer Architecture Matters

- **Maintainability** — change the UI without rewriting the business layer.
- **Reuse** — one service supports web, mobile and desktop.
- **Testability** — test a service without clicking through a graphical interface.
- **Adaptability** — swap MySQL for PostgreSQL at the data-access boundary.
- **Team development** — separate teams per layer with clear boundaries.

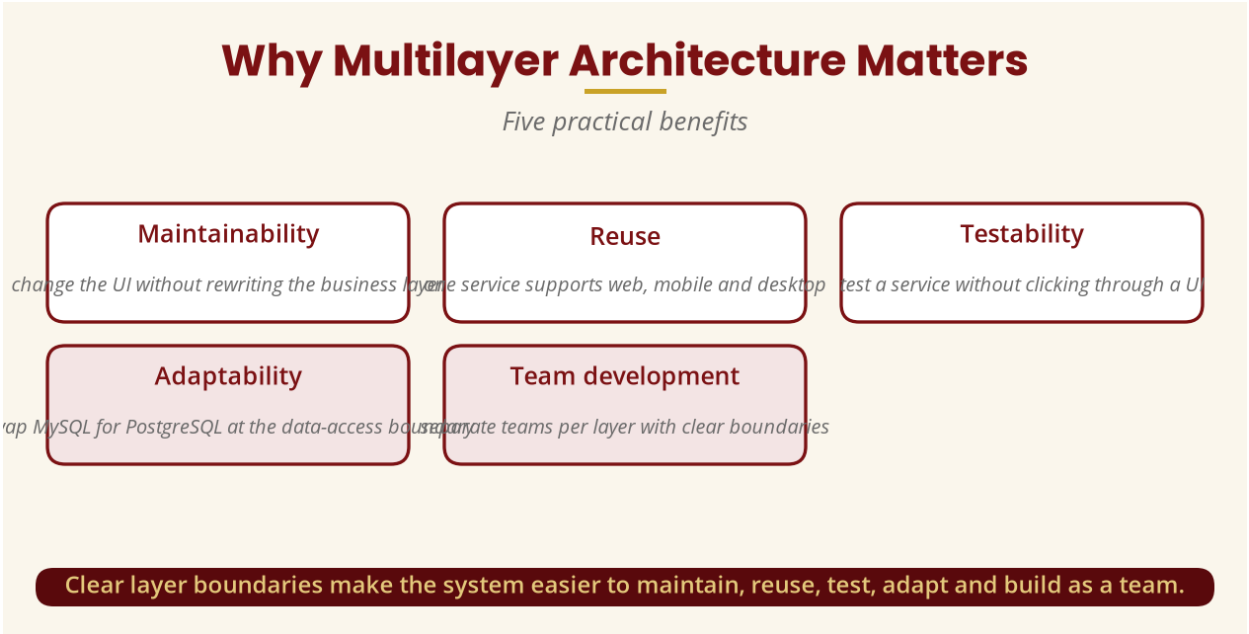

Figure 16 — Benefits of multilayer architecture

20. Limitations — More Layers Is Not Always Better

- **Additional complexity** — Controller -> Service -> Domain -> Repository -> Database introduces abstractions.
- **Performance overhead** — extra abstractions or network boundaries can cost time.
- **Overengineering** — a tiny app may not need 15 layers, 40 interfaces and 100 packages.
- **Incorrect layer responsibilities** — business logic scattered into controllers, repositories and UI looks layered but is poorly designed.

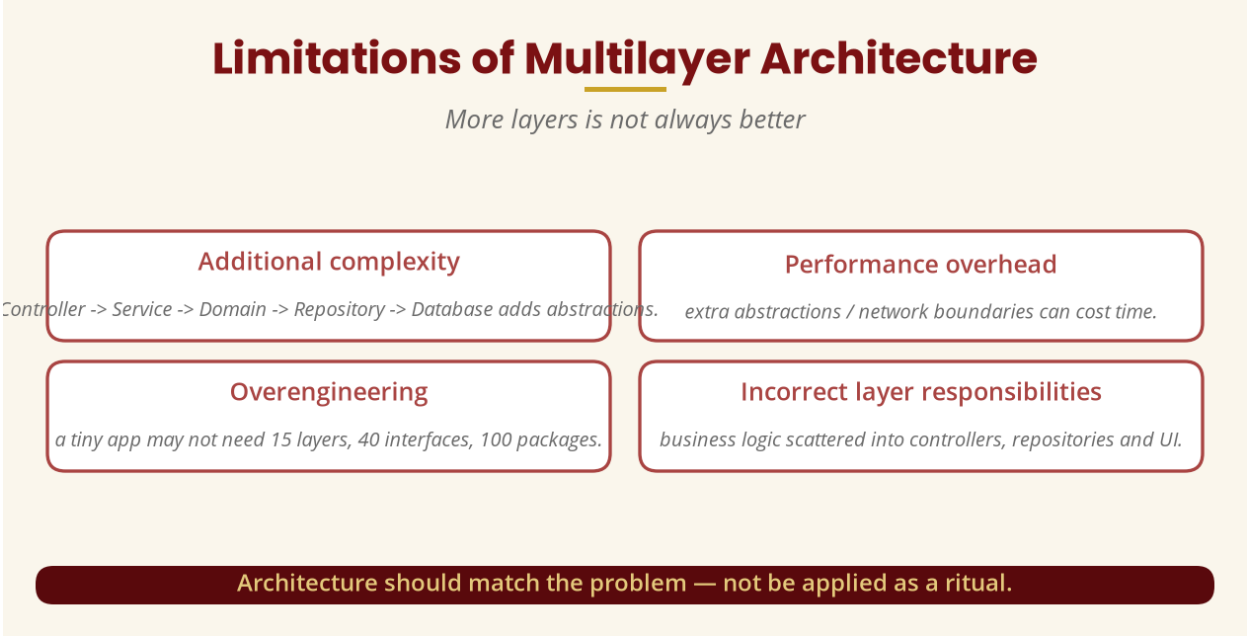

Figure 17 — Limitations of multilayer architecture

PART W & X — REUSE AND ADAPTABILITY CASE STUDIES

21. Component Reuse and Adaptability

Reuse case study: a university with a Registration, Notification, Authentication and Payment component later develops a mobile application — instead of rebuilding authentication, notification and payment from scratch, the mobile app interacts with the existing shared services. **Adaptability case study:** the university wants to notify students on successful registration — initially email, then SMS, then mobile push; a rigid architecture requires major changes, while an adaptable Notification abstraction (Email, SMS, Push) lets the application call `notifyStudent()` without being tied to one technology.

PART Y & Z — COMPONENT DIAGRAM EXERCISE AND NOTATION

22. Exercise and Notation

Exercise: model an Online Banking System with components Customer Interface, Authentication, Account Management, Transaction, Payment, Notification and Database. Important component-diagram elements to become familiar with: **component, interface, provided interface, required interface, dependency, connector, port**. The point is not merely drawing the interface but understanding *what functionality one component offers and what functionality another component requires*.

PART AB — DEPLOYMENT THINKING

23. Logical vs Deployment Architecture

Logical architecture describes how software is organised; **deployment architecture** describes where software executes. Example: Student's Phone (Mobile Application) -> Internet -> Application Server (Registration Service, Authentication) -> Database Server (University Database). A **UML deployment diagram** represents the runtime arrangement of software artifacts on nodes.

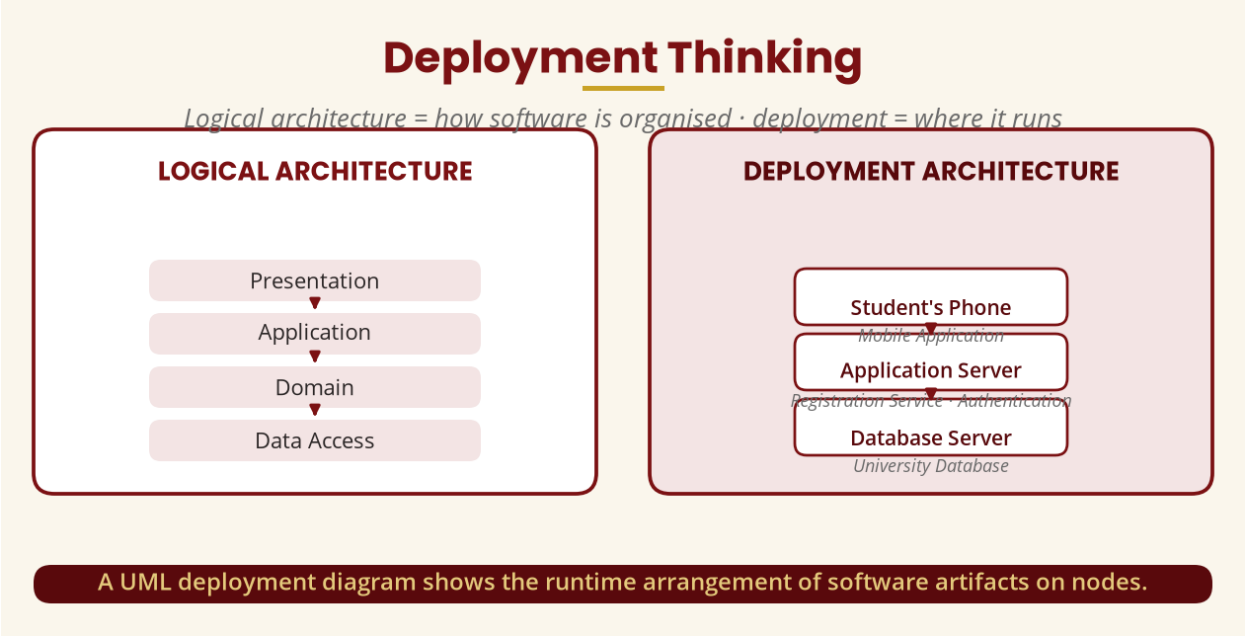

Figure 18 — Deployment thinking

PART AC & AD — REUSABILITY DESIGN PRINCIPLES

24. Principles and a Warning

- **Clear interfaces** — a component is easier to reuse through a well-defined interface.

- **Minimise unnecessary dependencies** — don't couple a component to one specific application.
- **Encapsulation** — expose processPayment() while hiding databaseConnection(), APIKeyHandling() and retryLogic().
- **Configuration** — make components configurable (channel = SMS) rather than hard-coded.

Warning: a component should not be made reusable simply because "reusable is always better." Spending three weeks turning a SmallCalculator into a huge framework may be unnecessary. Good design considers current requirements, likely changes, maintenance cost, complexity and reuse opportunities — **design for appropriate reuse, not reuse at any cost.**

25. Tutorial and Practical

Tutorial 1 — classify each activity into a layer (Student clicks Register -> Presentation; RegistrationService -> Application; Course business rule -> Domain; CourseRepository -> Data Access; Database storage -> Database; confirmation display -> Presentation). **Tutorial 2** — critique a StudentController containing HTML, SQL, password validation, registration, email, fees and PDFs: what is wrong, is cohesion high or low, which responsibilities should move, propose a better architecture. **Tutorial 3** — discuss the advantages and disadvantages of a shared authentication component/service for the web, mobile and desktop applications.

Practical (1 hour): extend the Week 12 system — (A) package diagram (Authentication, Academic, Registration, Finance, Notification); (B) component diagram (Web Application, Mobile Application, Authentication, Registration, Payment, Notification, Database with relationships); (C) layered architecture (Presentation, Application, Domain, Data Access, Database); (D) identify three reusable components and explain why; (E) explain how the system changes if a mobile application is introduced.

26. Common Student Confusions

- **Layer vs component** — a layer is a logical architectural level; a component is a modular software unit (the Domain Layer may contain Student, Course and Registration components).
- **Component vs class** — a class is a finer-grained building block; a component is a larger modular unit.
- **Package vs layer** — a package is an organisational namespace; a layer is an architectural separation by responsibility.
- **Reuse vs inheritance** — inheritance can support reuse, but reuse does not require it (composition, interfaces, services, libraries, frameworks).
- **High cohesion vs low coupling** — "keep related things together and unnecessary dependencies apart."

27. Week 13 Summary

Concept	Simple meaning
Software architecture	High-level organisation of a software system.
Layer	Logical grouping of related responsibilities.
Presentation / Application	User interaction / use-case coordination.
Domain / Data access	Business rules / persistence.
Separation of concerns	Different parts handle different responsibilities.
Component	Modular unit encapsulating functionality behind interfaces.
Component interface	The contract through which a component is used.
Reusable component	Designed for use in more than one context.
Adaptable component	Can be configured or extended for new situations.
Package	Organisational namespace/grouping of model elements.
Dependency	One element relying on another.
Deployment	Where the software actually executes.

THE CORE MESSAGE

A good OO designer does not simply produce many UML diagrams. The designer decides how **responsibilities, objects, components and layers** should work together, while keeping the system **understandable, reusable, adaptable and maintainable**. Aim for **high cohesion and appropriately low coupling**; design components for **appropriate reuse**, not reuse at any cost; and let **architecture match the problem**.

28. Examination-Style Questions

Essay (25 marks). Explain multilayer architecture in object-oriented design. Using a University Course Registration System, describe the responsibilities of the Presentation, Application, Domain and Data Access layers.

Essay (20 marks). Differentiate a UML package from a UML component, and explain how both are used when designing a large object-oriented system.

Essay (25 marks). A company's user interface directly accesses the database and contains most of the business rules. (a) Identify four problems. (b) Propose a multilayer architecture. (c) Explain how it improves maintainability and reuse.

Scenario (30 marks). Design a component-oriented architecture for an online banking system: major components, responsibilities, interfaces and dependencies, and explain how the design promotes reuse and adaptability.

29. References and Further Reading

Authoritative / current online sources

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1.* OMG — the current formal UML specification.
- Object Management Group (OMG). *What is UML?* Overview of UML structural, behavioural and interaction diagrams.
- Microsoft Support. *UML diagrams in Visio.* Guidance covering class, component, deployment, sequence, activity and state-machine diagrams.
- Microsoft Support. *Create a UML deployment diagram.* Deployment diagrams and the runtime arrangement of software artifacts.
- Sparx Systems. *Enterprise Architect UML Package Documentation.* UML packages and their role in organising model elements.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*